Pulire i dati

L3 · Informatica per economisti · SIMA 2026

Oggi

1

Spiegazione e demo

25 min

Testo, conversioni, valori mancanti

2

Task

60 min

Pulire imprese.xlsx, un problema alla volta

3

Debrief

25 min

Le trappole silenziose e le nostre decisioni

Pulire è decidere

1

Trova

il problema

2

Decidi

cosa fare, e perché

3

Correggi

con il codice

4

Verifica

conta prima, conta dopo

Nessuna correzione è automatica: ogni volta scegli tu. **Le decisioni vanno scritte.**

La regola: conta prima, conta dopo

PYTHON

```
print(df.shape)
df = df.drop_duplicates()
print(df.shape)
```

OUTPUT

```
(1000, 5)
(980, 5)
```

Quante righe dovevano sparire? 20? Allora va bene.

Molti errori di pulizia non danno messaggi: perdi dati in silenzio.

Contare è l'unica difesa.

Lavorare sul testo: .str

PYTHON

```
citta.str.strip()
```

OUTPUT

```
" Milano "  →  "Milano"
```

PYTHON

```
citta.str.strip().str.lower()
```

- Le colonne di testo hanno comandi dedicati, dopo .str
- strip, lower, upper, title, replace…
- Si possono mettere in fila, uno dopo l'altro
- **Quelli automatici non conoscono le eccezioni: controlla sempre**

I dizionari: coppie chiave → valore

PYTHON

```
capoluoghi = {"Lazio": "Roma",
              "Veneto": "Venezia"}
```

PYTHON

```
capoluoghi["Lazio"]
```

OUTPUT

```
"Roma"
```

PYTHON

```
df["citta"].replace({"Milan": "Milano",
                     "Rome": "Roma"})
```

- Tra parentesi graffe, coppie chiave: valore
- Con la chiave tra parentesi quadre leggi il valore
- Come un vocabolario: cerchi la parola, trovi la definizione
- **replace usa un dizionario come tabella di correzioni: sbagliato → giusto**

Da testo a numero

PYTHON

```
pd.to_numeric(col, errors="coerce")
```

OUTPUT

```
"12.5"   →  12.5
"abc"    →  NaN
"12,5"   →  NaN
```

ATTENZIONE

Quello che non si converte diventa un valore mancante. Senza errori.

Dopo ogni conversione: conta i mancanti.

Correggere solo alcune righe

PYTHON

```
negativi = df["prezzo"] < 0
print(negativi.sum())
df.loc[negativi, "prezzo"] = None
```

- Una condizione è una colonna di True e False
- .sum() conta quante righe la rispettano
- .loc[righe, colonna] = valore

MANCANTE NON È ZERO

Media di 10, ?, 20

```
con il mancante: 15
con lo 0:        10
```

Correggere una cella a mano

In Excel

clicchi sulla cella, scrivi 12, invio.
Nessuno saprà mai che l'hai fatto.

In Python

```python
riga = ((df["cliente"] == "C042")
        & (df["anno"] == 2021))
print(riga.sum())     # deve dare 1
df.loc[riga, "ordini"] = 12
```

1

Sempre nel codice

la correzione resta scritta e si può rifare

2

Per contenuto, non per numero di riga

dopo un ordinamento la riga 500 è un'altra

Due condizioni insieme: &, e ognuna tra parentesi.

Task: pulisci il dataset

Obiettivo: 2.500 righe, 20 regioni, un fatturato numerico e plausibile.

60 minuti · apri il notebook dal README

Debrief

Le trappole silenziose

L'ordine delle sostituzioni conta

Prima la virgola

`"1.032,9" → "1.032.9"`

1.621

valori mancanti

Prima il punto

`"1.032,9" → "1032,9" → "1032.9"`

50

valori mancanti

Stesso codice, stesse funzioni, nessun errore. Solo il conteggio dei mancanti ti salva.

Un fatturato di 24 milioni?

1.407

fatturato mediano

24.586.100

fatturato massimo

Cinque valori enormi, poi un salto netto.

Negli altri anni la stessa impresa fattura circa mille volte meno: inseriti in euro, non in migliaia.

Decisione: soglia 200.000, divisi per 1.000.

Il codice non sa che è un errore. Lo sai tu, perché conosci i dati.

Le scorciatoie automatiche

PYTHON

```
df["regione"].str.strip().str.title()
```

OUTPUT

```
Valle d'Aosta  →  Valle D'Aosta
Emilia Romagna →  Emilia Romagna
regioni: 21
```

.title() sistema le maiuscole, ma rompe Valle d'Aosta e non aggiunge il trattino.

Più sicuro: strip() e poi correzioni esplicite con replace().

Ce ne accorgeremo nella L6, quando uniremo i dati delle regioni.

Le nostre decisioni

Problema	Decisione	Verifica
15 righe duplicate	eliminate	2.500 righe
fatturato come testo	convertito, "n.d." → mancante	50 mancanti
5 fatturati in euro	divisi per 1.000	massimo plausibile
regioni scritte male	strip + correzioni esplicite	20 regioni
addetti negativi (IMP0253, 2019)	corretto a 12, su indicazione del collega	nessun negativo
valori mancanti	lasciati mancanti	isna().sum()

Questa tabella, in un lavoro vero, va scritta. Tra sei mesi non te la ricorderai.

Prossima volta: raggruppare e disegnare

Compito: Kaggle Learn – Pandas, «Data Types and Missing Values» (~20 min)